

O artigo “Hexagonal Architecture”, escrito por Alistair Cockburn, apresenta uma proposta interessante e prática para resolver problemas comuns no desenvolvimento de software, principalmente aqueles relacionados ao acoplamento excessivo entre partes do sistema. O autor parte de uma crítica às arquiteturas tradicionais em camadas, mostrando que, na prática, elas acabam misturando lógica de negócio com detalhes técnicos, como interface gráfica e banco de dados, o que dificulta manutenção e evolução do sistema.

O principal conceito apresentado é a chamada arquitetura hexagonal, também conhecida como “ports and adapters”. A ideia central é separar claramente o núcleo da aplicação, onde está a lógica de negócio, de tudo que é externo, como interfaces, bancos e APIs. Segundo o próprio artigo, o sistema pode ser visto como algo “interno” que se comunica com o mundo externo por meio de portas, sendo que essas portas representam interfaces bem definidas. Isso muda bastante a forma de pensar o sistema, porque deixa de ser algo dividido em camadas rígidas e passa a ser algo mais flexível e orientado a interações.

Um ponto que achei interessante é que o formato de hexágono não tem um significado matemático ou estrutural fixo. Ele é usado apenas como uma forma visual para mostrar que podem existir várias conexões com o mundo externo, e não apenas uma sequência linear como nas arquiteturas tradicionais. Isso ajuda a quebrar aquele pensamento de que sempre existe uma ordem fixa tipo interface, serviço e banco. Aqui, qualquer componente externo pode se conectar ao sistema desde que respeite as interfaces definidas.

Outro conceito importante é a ideia de portas e adaptadores. As portas são basicamente contratos, ou interfaces, que definem como o sistema se comunica com o exterior. Já os adaptadores são implementações concretas dessas interfaces, como um banco de dados específico ou uma API externa. Isso permite trocar facilmente tecnologias sem impactar a lógica principal do sistema. Por exemplo, seria possível trocar um banco SQL por NoSQL sem alterar a regra de negócio, o que é uma vantagem muito relevante na prática.

Além disso, o artigo enfatiza bastante a questão da testabilidade. Como a lógica de negócio fica isolada, ela pode ser testada sem depender de banco de dados ou interface gráfica. Isso facilita muito o uso de testes automatizados e práticas como TDD. Para quem está estudando engenharia de software, isso é importante porque conecta diretamente arquitetura com qualidade de código.

Na minha visão, um dos maiores benefícios dessa abordagem é a redução do acoplamento. Em arquiteturas tradicionais, é comum ver código de negócio misturado com detalhes de framework ou banco, o que dificulta mudanças. Já na arquitetura hexagonal, existe uma separação clara de responsabilidades, o que torna o sistema mais modular e mais fácil de evoluir. Isso também evita o

chamado “lock-in” tecnológico, já que o sistema não depende diretamente de uma tecnologia específica.

Por outro lado, também é possível perceber que essa arquitetura pode aumentar a complexidade do projeto, principalmente para quem está começando. A necessidade de criar interfaces, adaptadores e separar bem as responsabilidades pode gerar mais arquivos e mais organização inicial. Para projetos pequenos, isso pode parecer um excesso. Porém, para sistemas maiores ou que precisam evoluir ao longo do tempo, essa estrutura tende a compensar.

Outro ponto relevante é que essa arquitetura se conecta bem com outros conceitos modernos, como Domain-Driven Design e Clean Architecture. Todas essas abordagens colocam a lógica de negócio no centro e tentam proteger essa parte das mudanças externas. Isso mostra que o artigo do Cockburn foi bastante influente e continua sendo atual mesmo anos depois de ter sido escrito.

No geral, a leitura do artigo traz uma mudança de mentalidade importante. Em vez de pensar o sistema como uma sequência de camadas, passamos a enxergá-lo como um núcleo isolado que se comunica com o mundo externo por meio de interfaces bem definidas. Para mim, isso ajuda a entender melhor como projetar sistemas mais flexíveis, testáveis e fáceis de manter.